

Lösungsvorschlag

1. Aufgabe - Listen und Durchschnittsleistung

```
List<KuhData> erzeugeKuhData(List<ProdData> prodData) {
    List<KuhData> kuhData = new List<KuhData>();

    for (int i = 0; i < prodData.size(); i++) {
        ProdData p = prodData.get(i);
        KuhData kuh = new KuhData(p.getName(), 0.0);

        if (!kuhData.contains(kuh)) {
            kuhData.add(kuh);
        }
    }
    return kuhData;
}
```

```
List<KuhData> calcAvgLeistung(List<ProdData> prodData, Date start, Date ende) {
    List<KuhData> kuhData = erzeugeKuhData(prodData);

    for (int i = 0; i < kuhData.size(); i++) {
        double summe = 0.0;
        int anzahl = 0;

        for (int j = 0; j < prodData.size(); j++) {
            ProdData p = prodData.get(j);
            if (p.getDate().between(start, ende)
                && p.getName() == kuhData.get(i).getName()) {
                summe = summe + p.getLeistung();
                anzahl = anzahl + 1;
            }
        }

        if (anzahl > 0) {
            kuhData.get(i).setAvgLeistung(summe / anzahl);
        }
    }
    return kuhData;
}
```

2. Aufgabe - Klassenmodell und Polymorphie

```

AufgabenContainer
- aufgaben : List<Aufgabe>
+ aufgabeHinzufuegen(aufgabe : Aufgabe) : void
+ aufgabenHeute() : List<Aufgabe>

Aufgabe (abstrakt)
- beschreibung : String
- anzahlMitarbeiter : int
+ aufgabeBeschreiben() : String
+ heuteAusfuehrbar() : Boolean

AufgabeFixTermin erbt von Aufgabe
- termin : Date
+ heuteAusfuehrbar() : Boolean

Aufgabewetterabhaengig erbt von Aufgabe
- wetterVoraussetzung : Wetter
+ heuteAusfuehrbar() : Boolean

Beziehungen:
AufgabenContainer verwaltet 0..* Aufgabe-Objekte.
AufgabeFixTermin und Aufgabewetterabhaengig generalisieren/spezialisieren Aufgabe.
    
```

```

List<Aufgabe> aufgabenHeute() {
    List<Aufgabe> liste = new List<Aufgabe>();

    for (int i = 0; i < aufgaben.size(); i++) {
        Aufgabe a = aufgaben.get(i);
        if (a.heuteAusfuehrbar()) {
            liste.add(a);
        }
    }
    return liste;
}
    
```

Der gesuchte Mechanismus ist Polymorphie mit dynamischer Bindung. Obwohl die Liste den Typ Aufgabe enthält, wird zur Laufzeit anhand des tatsächlichen Objekttyps die passende überschriebene Methode heuteAusfuehrbar ausgeführt.

3. Aufgabe - Relationales Modell und Speicherbedarf

Tabelle	Attribute
Bodentyp	BodentypID (PK), Bezeichnung
Feld	FeldID (PK), BodentypID (FK), weitere Felddaten
Feldfrucht	FeldfruchtID (PK), Bezeichnung
Bodentyp_Feldfrucht	BodentypID (PK/FK), FeldfruchtID (PK/FK)
Taetigkeit	TaetigkeitID (PK), Beschreibung
Bearbeitungsschritt	BearbeitungID (PK), FeldID (FK), FeldfruchtID (FK), TaetigkeitID (FK), ZeitraumVon, ZeitraumBis

Speicherbedarf: 50.000.000 Pixel x 24 Bit = 1.200.000.000 Bit = 150.000.000 Byte je Bild. 500 Bilder x 52 Wochen = 26.000 Bilder. 26.000 x 150.000.000 Byte = 3.900.000.000.000 Byte. Umrechnung in TiB: 3.900.000.000.000 / 1024^4 = 3,55 TiB.

4. Aufgabe - SQL und CRUD

CRUD	DDL	DML	DQL
Create	CREATE	INSERT	-

AP2 Sommer 2026 - GA2 AE

Entwicklung und Umsetzung von Algorithmen - Musterlösung

Read	-	-	SELECT
Update	-	UPDATE	-
Delete	DROP	DELETE	-

```
SELECT
    tk.TK_Kategorie,
    COUNT(tb.TB_ID) AS AnzahlTiere,
    MAX(g.MaxGewicht) AS SchwerstesTier,
    MAX(TIMESTAMPDIFF(YEAR, tb.TB_GebDat, CURRENT_DATE)) AS AeltestesTier,
    AVG(TIMESTAMPDIFF(YEAR, tb.TB_GebDat, CURRENT_DATE)) AS DurchschnittAlter
FROM Tierkategorie tk
LEFT JOIN Tierbestand tb
    ON tb.TB_TKID = tk.TK_ID
    AND tb.TB_SchlachtDat IS NULL
LEFT JOIN (
    SELECT TZI_TBID, MAX(TZI_Gewicht) AS MaxGewicht
    FROM TierZusatzInfo
    GROUP BY TZI_TBID
) g ON g.TZI_TBID = tb.TB_ID
GROUP BY tk.TK_ID, tk.TK_Kategorie;
```

```
INSERT INTO Archiv_Tierbestand
    (A_TBID, A_Tierkategorie, A_ChipNr, A_GebDat, A_SchlachtDat, A_MaxGewicht)
SELECT
    tb.TB_ID,
    tk.TK_Kategorie,
    tb.TB_ChipNr,
    tb.TB_GebDat,
    tb.TB_SchlachtDat,
    MAX(tzi.TZI_Gewicht)
FROM Tierbestand tb
JOIN Tierkategorie tk ON tk.TK_ID = tb.TB_TKID
LEFT JOIN TierZusatzInfo tzi ON tzi.TZI_TBID = tb.TB_ID
WHERE tb.TB_SchlachtDat IS NOT NULL
GROUP BY tb.TB_ID, tk.TK_Kategorie, tb.TB_ChipNr, tb.TB_GebDat, tb.TB_SchlachtDat;

DELETE FROM TierZusatzInfo
WHERE TZI_TBID IN (
    SELECT TB_ID FROM Tierbestand WHERE TB_SchlachtDat IS NOT NULL
);

DELETE FROM Tierbestand
WHERE TB_SchlachtDat IS NOT NULL;
```